

CreditVault — Learn and Present Your Fraud Detection Project

Open the app · Detailed viva guide

This README is your study guide. Read it with the app open beside you. Here, a **page/tab** means one screen of the website; a **slide** means a page in your presentation. The website has nine tabs, and this guide explains all nine. A suggested slide order is included near the end.

You do not need to memorize the Python code first. Start by understanding the problem, the data, the inputs, the prediction, and how we measure mistakes.

Contents

CreditVault — Learn and Present Your Fraud Detection Project

1. Understand the project in one minute
 2. Five words to learn before opening the tabs
 3. The complete journey of a transaction
 4. Know which experiment you are presenting
 5. Tab 1 — Overview
 6. Tab 2 — Data Explorer
 7. Understand every PaySim input and calculated feature
 8. Tab 3 — Model Performance
 9. Tab 4 — Explainability
 10. Tab 5 — Live Predictor
 11. Tab 6 — Methodology
 12. Tab 7 — Glossary
 13. Tab 8 — Check a card payment
 14. Tab 9 — How it works & results
 15. A presentation slide plan you can follow
 16. Answers to questions you are likely to receive
 17. How the code and saved files fit together
 18. What to learn first, and how to check yourself
- Technical appendix — setup and reproducibility
- Run locally
- Folder structure
- Reproduce training and SHAP
- Streamlit Community Cloud

1. Understand the project in one minute

The problem: most transactions are legitimate, but a small number are fraudulent. A useful detector should catch fraud while avoiding too many false alarms.

Our solution: train machine-learning models on labeled examples, score a transaction, compare the score with a review threshold, and explain the model's decision.

What you can say:

"My project demonstrates a fraud detection pipeline: transaction data, understandable features, trained models, evaluation, and explanations. The main dashboard uses PaySim simulated financial transactions. It compares Logistic Regression, Random Forest, and XGBoost; the saved model is Random Forest. The app explains both its predictions and its measured mistakes."

Be clear about the title and dataset: your submitted topic is **Credit Card Fraud Detection System**. PaySim is **synthetic mobile-money transaction data**, not real credit-card purchase data. We use it as an educational financial-fraud example with understandable fields. Two preserved tabs use a separate synthetic credit-card experiment called Sparkov. Explain this distinction rather than calling all the data real card transactions.

2. Five words to learn before opening the tabs

Word	Meaning in this project	Example
Dataset	A collection of recorded examples.	A table of transactions.
Row	One example in the table.	One payment or transfer.
Column	One piece of information about each example.	Transaction Amount.

Word	Meaning in this project	Example
Feature	Information the model receives to make a prediction.	Amount, balances, or calculated balance mismatch.
Label / target	The known answer used to teach and evaluate the model.	Fraud or Legitimate.

A feature is an input; the label is the answer. The recorded fraud label is never given to the model when asking it to predict that transaction.

“Matrix” simply means a rectangular arrangement of numbers. The **confusion matrix** is a small results table; it is not an input a customer needs to enter. The main PaySim model uses readable features, not the original notebook’s anonymous PCA vectors.

3. The complete journey of a transaction

```

Recorded transaction details
↓
Calculate readable features
↓
Apply the saved preprocessing
↓
Trained Random Forest produces a fraud score
↓
Compare score with the saved review threshold
↓
Show decision, score band, and SHAP explanation

```

Training means learning patterns from earlier labeled examples.

Prediction, also called inference, means applying that already-trained model to a transaction. Opening the app or clicking Analyze transaction does not train a new model.

A **pipeline** packages the fitted preprocessing and classifier together. The saved model.pkl contains that pipeline. This helps the app use the same transformations and feature order used during training.

4. Know which experiment you are presenting

Item	Main PaySim experiment	Preserved Sparkov card experiment
Where it appears	First seven tabs.	Last two tabs.
Transactions	Simulated mobile-money transfers, deposits, withdrawals, and payments.	Simulated credit-card purchases.
Main information	Amount, transaction type, sender/receiver balances, earlier activity.	Purchase details and earlier card spending history.
Saved classifier	Random Forest.	Gradient Boosting.
Currency	PaySim's source currency units; do not invent a dollar/rupee conversion.	USD.
Explanation	Genuine SHAP values.	Numerical tree contributions, not SHAP.
Results	Its own chronological test period.	A different chronological test period.

Do not combine their metrics. If you move to a Sparkov tab, say: “This is the separate card-history experiment.” The old notebook/PCA model remains in the repository but does not power the main PaySim predictor.

5. Tab 1 — Overview

Purpose: introduce the problem, dataset size, fraud rarity, and what the system does.

Open **What am I looking at?** first. It gives a short explanation written for someone seeing the app for the first time.

Understand the four cards

Card	Meaning	Actual value
Source transactions	Every transaction in the original verified PaySim source.	6,362,620

Card	Meaning	Actual value
Recorded fraud	Transactions the simulator labeled as fraud.	8,213
Source fraud rate	Fraud count divided by source transaction count.	About 0.1291%
Project sample	The smaller collection retained for training/exploration.	500,000

The full source includes **6,354,407 legitimate transactions**. The project sample keeps **all 8,213 fraud rows** plus **491,787 legitimate rows**. Its fraud share is **1.6426%** because normal transactions were sampled down.

The 500,000-row sample includes different chronological periods. **The model does not train on all 500,000 rows**. It fits only the earlier training portion.

Understand the distribution chart

The bars compare legitimate and fraudulent counts. The vertical axis is **logarithmic**: equal distances represent multiplication rather than equal additions. This allows the tiny fraud class to remain visible next to millions of normal transactions. Read the printed counts; do not infer percentages from bar heights.

Why it matters: fraud is rare. A model can get high accuracy by calling everything legitimate and still catch no fraud.

Say aloud:

“The source contains about 6.36 million simulated transactions, with fraud in only about 0.129%. We retain every fraud case in a 500,000-row study sample. The dashboard separates sample statistics from evaluation on the full later test period.”

6. Tab 2 — Data Explorer

Purpose: understand what the transactions look like before discussing the model.

What each control does

Control	What happens when you change it
Recorded outcome	Shows legitimate rows, fraud rows, or both. These are known dataset labels.
Transaction Type	Includes only the selected action categories.
Maximum Transaction Amount	Removes rows above your chosen amount.
Rows to preview	Shows 10, 25, 50, or 100 matching rows in the table. It does not change training.

The **Matching transactions**, **Recorded fraud**, and **Filtered fraud share** cards describe the rows left after these filters. Their values are not the original source fraud rate or the model's accuracy.

Read the charts

Chart	What it tells you	How to explain it
Transaction types and recorded outcomes	Counts of fraud/legitimate rows within each selected type.	"This compares which types appear in the filtered sample." The count axis is logarithmic.
Transaction amounts	How amount patterns differ between the two labeled classes.	Each class is normalized separately, so the lines compare patterns, not total class size. The amount axis is logarithmic.
Activity across the day	Transaction counts at repeating simulation-hour indexes 0–23.	"These hours come from the simulator's counter, not a measured payment duration."
Correlation heatmap	How numeric columns move together in the filtered sample.	Hover over a cell to see its correlation value.

Correlation ranges from -1 to $+1$. Near $+1$ means two quantities tend to increase together; near -1 means they tend to move in opposite directions; near 0 means little linear relationship. Near 0 does not rule out a more complicated relationship. A blank cell can mean a feature has no variation in the filtered rows.

Correlation describes association; it does not prove causation. The recorded label can appear in this exploration chart but is excluded from the classifier inputs.

Try this: select only Fraud, inspect a few rows and transaction types, then restore both outcomes. Open **What does each displayed column mean?** to see the definitions.

Say aloud:

“This page helps me inspect and compare the data. Filtering and correlation are exploratory analysis; they do not train the model or prove that one feature causes fraud.”

7. Understand every PaySim input and calculated feature

Inputs you see in Live Predictor

Input	Plain meaning	What you should know
Transaction Type	The action performed.	Cash in adds money; Cash out withdraws it; Debit charges an account; Payment pays a merchant; Transfer moves money between accounts.
Transaction Amount	Money moved in this transaction.	PaySim currency units, not a promised currency conversion.
Sender Balance Before	Sender's recorded money before the transaction.	“Sender” is the account making the transaction.
Sender Balance After	Sender's recorded money after the transaction.	Available after the event.
Receiver Balance Before	Receiver's recorded money before the transaction.	A zero may represent an unavailable merchant balance.
Receiver Balance After	Receiver's recorded money after the transaction.	A zero does not automatically prove fraud or an empty real account.

Input	Plain meaning	What you should know
Transaction Hour	Numbered hour since the simulation started.	Source hours run from 1 to 743. It is not how long a transaction took.
Receiver Activity Count	Number of transactions received in strictly earlier simulation hours.	A bank would calculate this from history.
Sender Transactions in Last 24h	Sender's transaction count in the preceding 24 hours.	Same-hour and future transactions are excluded.

A customer usually cannot know the receiver's private balance or history. **This form demonstrates a bank-side analysis using saved simulated records.** Use recorded examples when those details are unknown; do not make up banking information and treat its prediction as a verified result.

Features the code calculates automatically

Feature	Formula / rule	What it means
Amount Debited from Sender	Sender Before – Sender After	How much the recorded sender balance decreased.
Amount Credited to Receiver	Receiver After – Receiver Before	How much the recorded receiver balance increased.
Sender Balance Mismatch	Sender Before – Amount – Sender After	Difference from the balance expected after a simple outgoing debit.
Receiver Balance Mismatch	Receiver Before + Amount – Receiver After	Difference from the balance expected after a simple incoming credit.
Account Emptied	1 if Sender After is zero and Amount is at least 95% of Sender Before; otherwise 0.	Whether the record satisfies this specific rule.
Amount as % of Sender Balance	$100 \times \text{Amount} / \text{Sender Before}$	How large the amount is relative to the starting balance.
Hour of Day	Transaction Hour modulo 24	Repeating simulation-hour index from 0 to 23.

Receiver Activity Count and Sender Transactions in Last 24h are also features. Transaction Type is encoded into yes/no category indicators inside the pipeline. The classifier starts with **15 readable features**, which become **19 numerical inputs** after one-hot encoding: 14 numeric features and five type indicators.

There is **no transaction duration column**. At a zero sender starting balance, the percentage is undefined; the pipeline fills that missing value using the training median. A negative debit can describe a balance increase. A mismatch is a calculated relationship, not an automatic accusation; deposits and unavailable balances need context.

Work through a simple transaction yourself

This is a **hypothetical arithmetic example**, not a saved model prediction:

- Type: Transfer; Amount: 1,000.
- Sender Before: 10,000; Sender After: 9,000.
- Receiver Before: 2,000; Receiver After: 3,000.
- Transaction Hour: 50.

Calculated feature	Calculation	Answer
Sender debit	$10,000 - 9,000$	1,000
Receiver credit	$3,000 - 2,000$	1,000
Sender mismatch	$10,000 - 1,000 - 9,000$	0
Receiver mismatch	$2,000 + 1,000 - 3,000$	0
Amount percentage	$100 \times 1,000 / 10,000$	10%
Account Emptied	Sender After is not zero.	0 / No
Hour of Day	Remainder after dividing 50 by 24.	2

If Sender After were 8,000, sender mismatch would become 1,000. That tells us the simple expected and recorded balances differ. **The trained model still makes the prediction.** This calculation alone does not tell us the true fraud label or score.

Columns excluded from prediction

Sender/Receiver Account Identifiers are used temporarily to group earlier activity, then removed. **Existing Rule Flag** is another rule's fraud alert, so it is excluded to avoid copying that rule's decisions.

Fraud / Legitimate is the known answer used during learning/evaluation, not an input for scoring.

8. Tab 3 — Model Performance

Purpose: show evidence that the model works on later transactions, including its mistakes.

This page evaluates **all 1,248,736 transactions in the later test period**, including **4,250 fraud cases**. It does not evaluate only the 12 demo cases or the fraud-enriched 500,000-row sample.

Understand the confusion matrix first

Read **rows as actual recorded outcomes** and **columns as the model's decisions**:

Actual outcome	Not flagged	Flagged for review
Legitimate	1,244,486 — correct legitimate decisions (TN)	0 — false alarms (FP)
Fraud	27 — missed fraud (FN)	4,223 — caught fraud (TP)

“Positive” means the fraud class. **True positive** means correctly flagging fraud; **false positive** means flagging a legitimate transaction. **False negative** means missing fraud; **true negative** means correctly leaving a legitimate transaction unflagged.

Now understand each metric

Metric	Question it answers	Saved Random Forest result
Precision	Of all transactions flagged, how many were actually fraud?	100.0000%
Recall	Of all actual fraud cases, how many did we catch?	99.3647%
F1 score	How well do precision and recall balance?	99.6813%

Metric	Question it answers	Saved Random Forest result
PR-AUC	How well does precision balance with recall across score cutoffs?	0.99999459
ROC-AUC	How well do scores rank fraud ahead of legitimate cases?	0.99999998

With these results:

```
Precision = TP / (TP + FP) = 4,223 / (4,223 + 0) = 100%
Recall = TP / (TP + FN) = 4,223 / (4,223 + 27) ≈ 99.3647%
F1 = 2 × Precision × Recall / (Precision + Recall) ≈ 99.6813%
```

Use proportions such as 1.0 and 0.993647 in the F1 formula. Precision is about **how trustworthy the alerts are**; recall is about **how much fraud we catch**. Here precision is 100% but recall is below 100%, so the model still missed fraud.

Accuracy asks what fraction of all decisions were correct. In the full source, predicting legitimate for everything gives about **99.87% accuracy and zero fraud caught**. That is why accuracy is secondary. The majority baseline displayed inside Model Performance is computed for its own test population; it need not equal the full-source percentage exactly.

Understand the threshold slider

The model outputs a score from 0 to 1. The **threshold** is the cutoff for flagging a transaction. The saved cutoff is **0.9175433179959634**, approximately **91.7543%** when shown on a percentage scale.

Changing the slider reclassifies the saved test scores; it does not retrain. These actual test counts illustrate the trade-off:

Threshold	Fraud caught	Fraud missed	False alarms
Saved cutoff, about 0.917543	4,223	27	0
0.900	4,224	26	0
0.100	4,250	0	257

At a lower cutoff, more records qualify for review. We may catch more fraud but create more false alarms. A higher cutoff can reduce alerts but miss fraud. A particular small threshold change may leave some counts unchanged.

The slider changes the deployed model's metric cards and confusion matrix. **It does not change the Live Predictor cutoff**, the model comparison table's saved cutoffs, or the learned model. The curves and their AUC values summarize many cutoffs, so their area does not change just because you move this single cutoff.

Read the curves and comparison table

Precision-recall curve: horizontal axis is recall; vertical axis is precision. It shows the trade-off as the cutoff varies. The dotted line marks the test fraud prevalence. Better curves generally keep precision high as recall increases. PR-AUC summarizes the area; closer to 1 is better.

ROC curve, inside the accuracy/ROC expander: horizontal axis is false-positive rate among actual legitimate transactions; vertical axis is recall among actual fraud. The diagonal is random ranking. Good curves approach the top-left. A high ROC-AUC alone can still hide an operationally large number of false alarms when legitimate transactions are numerous.

Average precision (AP) is a related but different precision-recall summary. This app's PR-AUC uses trapezoidal integration; AP uses precision over increases in recall. Do not call them the same number.

The comparison table uses the same full later test period, with each candidate's own validation-selected cutoff:

Model	Precision	Recall	F1	PR-AUC	ROC-AUC
Logistic Regression	95.6385%	50.0471%	65.7090%	0.79981591	0.99404533
Random Forest — deployed	100.0000%	99.3647%	99.6813%	0.99999459	0.99999998
XGBoost	100.0000%	99.5294%	99.7642%	0.99999196	0.99999997

Random Forest and XGBoost tied on validation F1 and AP. The predefined tie order chose Random Forest before looking at test performance. XGBoost's slightly higher test F1 did not change that selection. Choosing a new winner from the final test would use the test as another tuning set.

Say aloud:

"The saved model caught 4,223 of 4,250 simulated fraud cases and missed 27, with no false alarms in this test period. These are strong simulation results, not a promise that every real-world fraud will be caught."

9. Tab 4 — Explainability

Purpose: explain why the trained model gave a score, rather than showing only a result.

SHAP means **SHapley Additive exPlanations**. It assigns numerical contributions to the model's input features relative to a reference. It explains the classifier; it is not a separate classifier.

Global SHAP summary: many transactions

- Each row is a readable feature name. Features near the top have larger average absolute influence within this sample.
- Each dot describes that feature's contribution for one explained transaction.
- A dot to the right of zero increases the fraud score; a dot to the left decreases it.
- For numeric features, red means a higher feature value and blue means a lower one. **Red does not mean the transaction is fraud.** Transaction Type is a category, so do not interpret it as an ordered high/low number.
- The numerical influence table averages absolute contribution sizes. It shows influence magnitude, not whether a feature always raises risk.

This plot uses **300 actual later transactions: 299 legitimate and 1 fraud**. It is a small uniform explanation sample, not the entire test set or a balanced fraud sample. It mainly shows typical normal

transactions; use individual fraud examples to examine fraud decisions.

Waterfall: one transaction

Choose **Recorded transaction example**. The waterfall starts at a background reference score and adds feature contributions to reach that transaction's score. The background uses **100 actual training rows**.

For the selected Random Forest, contributions use **probability units**. Red bars increase its score; blue bars decrease it. The numerical table lets you check:

Reference value + sum of feature contributions = model output

For learning only, suppose a reference were 0.10 and two contributions were +0.15 and −0.05. The resulting score would be 0.20. Those are illustrative numbers, not a real saved transaction's explanation. Actual transactions use every contribution, including any remaining features grouped by the plot.

The plain-English reason uses the three largest contributions by absolute size. Some can raise the score and others can lower it. One-hot category contributions are combined into the single readable Transaction Type feature.

A large SHAP value means **this model relied on the feature**, not that the feature caused fraud. Related balances and mismatch features can share influence. A +0.05 contribution changes the model score by five percentage points relative to its reference; it does not prove five percentage points of real-world fraud risk.

Say aloud:

"SHAP helps me describe the model's decision. I can show which features increased or decreased its score, and the contributions reconstruct the output. This explains model behavior rather than proving someone's intent."

10. Tab 5 — Live Predictor

Purpose: demonstrate the actual saved model scoring one transaction.

What to do, step by step

1. Choose **Start with a recorded example**. The app fills in genuine saved transaction details.
2. Read the type, amount, and before/after balances. Expand **Timing and account activity** to inspect the hour and history counts.
3. Click **Analyze transaction**. The app calculates features and applies the saved pipeline.
4. Read **Flagged for review** or **Not flagged by the model**.
5. Read the gauge, the Low/Medium/High score band, and the three-feature reason sentence.
6. Open **Calculated model inputs · explained** to see what went into the classifier.
7. Open **Show the real per-transaction SHAP waterfall** to explain the score in detail.

What the result means

Output	Meaning
Model fraud probability	The classifier's fraud-class score, displayed as a percentage.
Review threshold	The saved cutoff at which the app flags a transaction.
Low	Score below half the saved threshold: below approximately 45.8772%.
Medium	Score from half the threshold up to it: approximately 45.8772% to below 91.7543%.
High	Score at or above the saved threshold: approximately 91.7543% or higher.
Reason sentence	The three strongest SHAP contributions for this transaction.
Recorded dataset outcome	The simulator's known answer for an unchanged saved example.

The score has **not been separately calibrated** to a real bank's fraud rate. "Calibrated" would mean checking whether, for example, transactions scored 70% really are fraud around 70% of the time in the relevant population. We have not established that. Score bands are presentation policies, not guaranteed real-world risk categories.

Changing any saved input creates a **hypothetical transaction**. Its true outcome is unknown, so the app does not reuse the saved example's label as the answer. A prediction is different from ground truth.

There are **12 saved examples**, including caught fraud, correctly unflagged legitimate transactions, and missed fraud. There are no saved false-alarm cases at the default cutoff because none occurred in that test evaluation. The example numbers are not a fraud ranking.

Try this: replay one example unchanged and compare its prediction with its recorded outcome. Then change an amount, submit again, and notice that the record is now hypothetical. Do not expect every edit to change the result; the model uses multiple features.

Say aloud:

"This is a completed-transaction demonstration. The bank would normally supply balances and earlier account activity. The app returns a model decision and its explanation; it does not connect to a bank, block a payment, or guarantee that a payment is safe."

11. Tab 6 — Methodology

Purpose: explain how the experiment was built and why its results are measured fairly.

Data preparation and time order

The process verifies the source, derives earlier activity using account identifiers, chooses whole-hour time boundaries, and samples legitimate transactions within transaction-type/hour groups. Fraud stays in its original chronological period. Only the earlier sampled training rows are fitted.

Period	Simulation hours	Full-source rows	Fraud rows	Purpose
Training	1–281	3,820,599	3,193	Learn model parameters from 298,640 sampled earlier rows.
Validation	282–355	1,293,285	770	Select the model and its cutoff using the full middle period.
Test	356–743	1,248,736	4,250	Measure performance on the full later period after selection.

These periods target roughly 60%/20%/20% of source rows; whole-hour boundaries mean they are not exact percentages. Transactions in the same hour never cross a split boundary. Earlier history can inform a later transaction, but later transactions and labels cannot inform an earlier one. This is a time split, not a claim that every test account is unseen.

Training is like studying; validation is like choosing your approach using a practice exam; testing is like the later exam used to report performance. Repeatedly choosing changes based on the final test would weaken its independence.

Why sampling and class weights are used

Class imbalance means legitimate examples greatly outnumber fraud. Keeping all fraud and sampling legitimate rows makes training manageable. **Stratified sampling** means preserving representation across groups—in this case legitimate transaction type/hour groups. Seed 42 makes the draws repeatable.

Class weights make mistakes on rare fraud examples matter more during fitting. Random Forest uses balanced weights in each tree's sample; Logistic Regression uses balanced weights; XGBoost uses a

fraud-class weight based on training class counts. Validation and test remain naturally imbalanced.

The new experiment does **not** use SMOTE or create invented fraud rows. Only 1,105 full-source transactions have any sender activity in the prior 24 hours, so this history feature is sparse. Its inclusion does not prove that it is important.

Preprocessing: prepare inputs without changing their meaning

Technique	Simple explanation	Used here
Feature engineering	Calculate useful relationships from existing information.	Debits, credits, mismatches, amount percentage, and activity.
One-hot encoding	Turn a category into separate yes/no indicators.	Transaction Type becomes five indicators.
Median imputation	Fill an undefined numeric value using the middle training value.	For example, amount percentage at zero starting balance.
Scaling	Put numerical inputs on comparable scales.	Logistic Regression baseline only; fitted on training data.
Leakage prevention	Keep information that would unfairly reveal the answer out of learning/prediction.	Exclude labels/rule flags; exclude future/same-hour history; fit preprocessing on training only.

For a Transfer, one-hot encoding sets the Transfer indicator to 1 and the other type indicators to 0. This avoids inventing a ranking such as “Transfer is greater than Payment.”

Explain the three models

Model	How it works in plain English	Why include it
Logistic Regression	Learns weighted inputs and transforms their combination into a classification score.	A simpler baseline. Despite its name, it can classify Fraud vs Legitimate.

Model	How it works in plain English	Why include it
Random Forest	Builds many decision trees using varied training samples and combines their fraud-class predictions.	Learns combinations of features and reduces reliance on a single tree.
XGBoost	Adds trees in sequence, using each new tree to improve the current model's predictions.	A strong boosting comparison for tabular data.

A **decision tree** learns a sequence of questions, such as comparisons with a balance mismatch or amount. Its final branches return learned scores. The example question is an explanation of the method, not a claim about an exact branch of our fitted model.

The selected Random Forest has **100 trees**, maximum depth **14**, at most **256 leaves per tree**, and at least **5 training examples per leaf**. A leaf is a terminal prediction branch. These restrictions limit tree complexity. Its fraud probability averages the trees' class-probability outputs; it is not simply the percentage of trees making a hard fraud vote.

The models are chosen by **validation F1**, with validation average precision as a tie-breaker. Exact ties use the predefined model order. Parameters are fixed settings for this experiment; we do not claim an exhaustive search or that these values are optimal for every dataset.

Say aloud:

"I keep the experiment in time order, fit preprocessing on training data, and select the model and threshold on validation. The full later test period measures the frozen choices. I address rare fraud through sampling and class weights and keep future information out of history features."

12. Tab 7 — Glossary

Purpose: a reference page for feature meanings and ML terminology. Open an unfamiliar term's expander while learning; definitions do not alter the model.

Term	Plain explanation
Supervised learning	Learning from examples that already have known answers.
Binary classification	Choosing between two classes: fraud and legitimate.
EDA	Exploratory Data Analysis: inspect data, counts, distributions, and relationships.
Baseline	A simple reference used to judge whether a more complex model helps.
Classifier	A model that predicts a class or class score.
Parameter / hyperparameter	A parameter is learned during fitting; a hyperparameter is a setting chosen for the learning procedure, such as tree count.
Overfitting	Learning training-specific details that do not generalize well to new examples.
Generalization	Performance on examples beyond those used to fit the model.
Threshold	A cutoff converting a continuous score into a review decision.
Calibration	Agreement between reported probabilities and observed outcome frequencies.
Precision / recall / F1	Alert correctness, fraction of fraud caught, and their harmonic-mean balance.
PR-AUC / ROC-AUC	Summaries of precision-recall performance and ranking across thresholds.
Confusion matrix	Four counts: correct normal, false alarm, missed fraud, and caught fraud.
SHAP	Feature contributions explaining a trained model's output relative to a reference.
Drift	Changes in transaction or fraud patterns over time that can reduce performance.

Term	Plain explanation
joblib	A Python tool used to save and reload the fitted model pipeline.
Streamlit	The framework that turns Python functions into this interactive website.

The app's Glossary contains the requested model and metric definitions; some additional study terms above help explain the overall project.

13. Tab 8 — Check a card payment

Purpose: demonstrate the preserved, separate **Sparkov synthetic credit-card model**.

Choose a sample card and saved payment. The editable fields are **Payment amount (USD)**, **Purchase category**, **Merchant**, **Payment date**, and **Payment time**. Click **Check payment** to see the score, review decision, explanation, and earlier history context.

You do not type anonymous vectors. Earlier history is calculated for that sample card automatically. The model's 13 features mean:

Feature or pair	Meaning
Payment amount	Size of this purchase.
Time of day: sine and cosine	Two related numerical representations that keep 23:59 close to 00:00. These count as two features.
Day of week	Weekday of the purchase.
Merchant category	Kind of purchase, such as groceries or travel.
Earlier average payment	Average spending amount before this transaction.
Amount / earlier average	Size of this purchase compared with the card's earlier mean.
Payments in previous hour / previous 24 hours	Earlier activity counts; these are two features.
Minutes since previous payment	Gap since the latest strictly earlier payment; not payment duration.

Feature or pair	Meaning
Merchant used before	Whether the card previously paid that merchant.
Category used before	Whether the card previously used that purchase category.
Earlier history available	Whether there is any earlier history to use.

Card identifiers group history but do not enter the classifier. Merchant name supports the familiarity check; the exact name is not directly fed into the estimator. Current transactions, timestamp ties, future transactions, and labels are excluded from the historical features.

The saved model is **HistGradientBoostingClassifier**, labeled Gradient Boosting in this experiment, with its own threshold of about **96.5891%**. Its explanations use actual additive numerical tree contributions in **log-odds**, not PaySim SHAP. Log-odds is another scale for expressing a probability; the model converts it to a score for display. Do not read a contribution on this scale as a direct percentage-point change.

Say aloud:

“This separate experiment uses synthetic card purchases and earlier spending patterns, instead of PaySim sender/receiver balances. The user enters familiar payment details, and the app derives the history features.”

14. Tab 9 — How it works & results

Purpose: show the methodology, comparison, and mistakes for that separate Sparkov model.

Its source portion contains **555,719 simulated credit-card transactions, 2,145 fraud cases, and 924 sample accounts**. This project creates its own chronological periods within that source portion. Its later test contains **107,706 payments, including 176 fraud cases**.

Metric / outcome	Sparkov Gradient Boosting result
Precision	70.4301%

Metric / outcome	Sparkov Gradient Boosting result
Recall	74.4318%
F1	72.3757%
Average precision	0.75837098
ROC-AUC	0.99600497
Fraud caught / missed	131 / 45
False alarms	55
Correct legitimate decisions	107,475

Read its confusion matrix the same way as PaySim's, but use these different counts. The model comparison includes Logistic Regression, Decision Tree, Random Forest, and Gradient Boosting. The history comparison reports F1 of about **66.86% for payment details only** versus **72.38% with earlier history** in this experiment.

That comparison investigates whether history helped this model here. It does not promise that history always helps. Some development test results were inspected before the final recipe; this is not claimed to be a completely untouched external audit. Open the page's dataset and limitations expanders for that context.

Say aloud:

"The card model makes both false alarms and misses. Its ROC-AUC is high, but its precision and recall show why we must inspect actual fraud outcomes. These are different data and results from the main PaySim study."

15. A presentation slide plan you can follow

These are suggested slides, not an existing PowerPoint file. Use the website for the demonstrations.

Slide	Put on the slide	Explain in your own words	App page to show
1. Title and objective	Project title, name, and goal.	Identify fraud while controlling false alarms; explain predictions.	Overview
2. Problem	Fraud is rare; mistakes have costs.	Missing fraud and disturbing legitimate customers are different errors.	Overview
3. Dataset	PaySim source, counts, synthetic scope.	Financial-fraud proxy; not real-card validation.	Overview / Methodology
4. Transaction fields	Amount, type, both accounts' balances, timing/history.	Each row is one transaction; label is the known answer.	Data Explorer
5. Feature engineering	One worked debit/mismatch/p percentage example.	We calculate understandable signals from existing data.	Methodology / Live Predictor inputs
6. Preparation and split	Encoding, missing values, earlier/middle/late r periods.	Preprocessing learns from training; validation selects; testing evaluates.	Methodology
7. Models	Logistic Regression, Random Forest, XGBoost.	Baseline, many trees combined, and sequential boosting; explain selection.	Methodology / comparison table
8. Evaluation	Precision, recall, F1, curves, actual confusion matrix.	Caught 4,223 fraud; missed 27; explain threshold trade-offs.	Model Performance
9. Explainability	SHAP global summary and one waterfall.	Which inputs changed the model score and in which direction.	Explainability
10. Demo	One unchanged example, prediction, known outcome, then an edit.	Recorded answer vs hypothetical prediction; score is not certainty.	Live Predictor

Slide	Put on the slide	Explain in your own words	App page to show
11. Card-history extension	Optional separate Sparkov model.	Explain its different inputs and lower measured results without merging them.	Last two tabs
12. Limits and future work	Synthetic data, completed-transaction scope, calibration, monitoring.	Appropriate real-card data and bank integration would be separate work.	Methodology

For a short demonstration: **Overview → Data Explorer → Methodology → Model Performance → Explainability → Live Predictor**. Use Glossary for questions and the card tabs as a clearly labeled extension. Restore the Model Performance slider to the saved default after exploring it.

16. Answers to questions you are likely to receive

What dataset did you use? PaySim Synthetic Financial Datasets For Fraud Detection for the main experiment; a separate preserved Sparkov credit-card experiment in the last two tabs.

Why synthetic data? Real banking records are private and difficult to obtain. Synthetic labeled examples support a student experiment, but simulated patterns do not establish real-world performance.

What is the selected model? Random Forest for PaySim, selected on validation before the final test. Gradient Boosting for the separate Sparkov experiment.

Where is the machine learning if you calculate mismatch formulas? The formulas create features; the trained classifier learns their relationships to labeled fraud. The formulas are not a complete hand-written fraud decision rule.

Why Random Forest when XGBoost has slightly higher test F1? They tied on validation F1/AP; the fixed tie order selected Random Forest. Final test performance did not select the winner.

Can a customer use this before paying? The main PaySim model needs after-balances, so it analyzes a completed transaction. Its form is a demonstration, not a pre-payment safety check. Banks would provide the required records and history.

Why don't I enter a card number? The demonstration does not connect to a bank. Identifiers select/group synthetic history; they are not predictive numeric features or a way to access a real account.

Why isn't 100% precision a perfect model? Recall is below 100%: 27 actual fraud cases were missed. The result also belongs to a particular synthetic test period, not all future transactions.

Does 0.8 mean there is a proven 80% chance of fraud? No. It is the fitted model's score; this experiment has not calibrated it to real deployment prevalence.

Does SHAP prove why fraud happened? No. It describes how the model's inputs contributed to a prediction relative to its reference, not causal evidence or someone's intent.

What did you use SMOTE for? The main experiment does not use it. It keeps actual fraud, samples legitimate training rows, and uses imbalance weights.

Does changing a number create a new correct answer? No. It creates a hypothetical record with an unknown true label. The app can predict it, but cannot validate that prediction without ground truth.

Why can we not just deploy this at a bank? A bank would need appropriate real transaction data, permissioned integration, calibration, security controls, human review, and ongoing validation. The current site is an educational ML demonstration.

17. How the code and saved files fit together

File / tool	Job
app.py	Starts the current dashboard.
presentation_dashboard.py	Builds the seven PaySim pages and embeds the two preserved card pages.
presentation_core.py	Shared feature formulas, readable names/glossary, input checks, scoring, and metrics.

File / tool	Job
<code>presentation_explain.py</code>	Real SHAP calculations, saved global explanations, and reason sentences.
<code>prepare_presentation_model.py</code>	Verifies source data, prepares history/samples/splits, trains candidates, and exports results.
<code>presentation_artifacts/model.pkl</code>	Selected fitted preprocessing + Random Forest pipeline.
<code>metadata.json</code>	Dataset source, feature recipe, model settings, selection, thresholds, and measured results.
<code>feature_columns.json</code>	Exact readable-source feature order used by the model.
<code>sample_data.parquet</code>	Saved 500,000-row table for exploration. Parquet is a compact table format.
<code>evaluation_scores.npz</code>	Full later test labels and candidate scores for metrics/slider behavior. NPZ is a compressed NumPy array archive.
<code>demo_cases.csv</code>	Genuine small test examples to replay. CSV is a text table.
SHAP CSV / NPZ / JSON assets	Training reference rows, explanation examples, actual contributions, and verification information.
<code>card_core.py</code> / <code>card_dashboard.py</code> / <code>card_artifacts/</code>	Separate card-history experiment, model, and UI.
<code>requirements.txt</code>	Python packages required to run the app.
<code>tests/</code>	Automated checks for feature calculations, inputs, metrics, explanations, and dashboard behavior.

Pandas handles data tables; NumPy handles numerical arrays; scikit-learn provides preprocessing, metrics, and the baseline/forest; XGBoost provides boosting; SHAP provides explanation values; Plotly draws interactive charts; Matplotlib draws SHAP plots; Streamlit builds the app; joblib saves/loads pipelines. GitHub stores files and Streamlit Community Cloud hosts the website.

`@st.cache_resource` avoids reloading the same model unnecessarily. `@st.cache_data` reuses already-loaded tables/scores. Lazy tabs load heavier content only when opened. These improve responsiveness;

they do not change the learned predictions.

18. What to learn first, and how to check yourself

Study in this order: **problem** → **dataset** → **inputs/formulas** → **training/validation/test** → **models** → **confusion matrix/metrics** → **threshold** → **SHAP** → **live demo** → **limitations**.

You are ready to present when you can answer these without reading:

- What is one row, one feature, and one label?
- Why is PaySim called synthetic, and why should you not call it real card data?
- Can you calculate the sender mismatch for a simple transfer?
- Which data teaches the model, which chooses the cutoff, and which measures results?
- What is the difference between precision and recall?
- Where are the 27 missed fraud cases in the confusion matrix?
- Why can lowering a cutoff create false alarms?
- What does a red SHAP dot mean, and what does it not mean?
- Why is a prediction not the same as a recorded outcome?
- Why are the last two tabs a separate experiment?

Start with understanding, then practice the short “Say aloud” explanations in your own words. Use [the detailed presentation/viva guide](#) for further questions and source details.

Technical appendix — setup and reproducibility

The sections below are for running or rebuilding the code after you understand the project.

Run locally

Use **Python 3.12** and the pinned model dependencies:

```
git clone https://github.com/me-vishnurnair/Machine-learning-project.git
cd Machine-learning-project
python -m venv .venv
source .venv/bin/activate
python -m pip install -r requirements.txt
streamlit run app.py
```

On Windows PowerShell, activate with `.venv\Scripts\Activate.ps1`. Open Streamlit's printed address. All runtime model, data, evaluation, and SHAP assets are committed; no training, full-source download, secrets, or extra upload is needed.

```
python -m pip install -r requirements-dev.txt
python -m pytest -q
```

`PRESENTATION_ARTIFACT_DIR` optionally selects another matching trusted local export. Models use `@st.cache_resource`; data and score loading use `@st.cache_data`. Native lazy tabs load the large data sample and SHAP plots only when needed. The app handles missing assets, invalid values, and empty filters with readable messages and never loads uploaded model pickles.

Folder structure

```

Machine-learning-project/
├─ app.py                                # Main Streamlit entry point
├─ presentation_dashboard.py             # One renderer per explained tab
├─ presentation_core.py                  # Names, glossary, exact feature recipe,
metrics
├─ presentation_explain.py              # Real SHAP, aggregation, and reason
sentences
├─ prepare_presentation_model.py        # Verified source, sampling, time split,
training
├─ requirements.txt
├─ requirements-dev.txt
├─ README.md
├─ .streamlit/config.toml
├─ assets/dashboard.css
├─ presentation_artifacts/
│   └─ model.pkl                        # Selected fitted preprocessing + Random
Forest
│   └─ metadata.json                   # Source, recipe, selection, full
evaluation
│   └─ feature_columns.json            # Exact fifteen-feature order
│   └─ requirements-model.txt
│   └─ sample_data.parquet             # All 500,000 study rows, no account IDs
│   └─ evaluation_scores.npz           # Full later-period labels and all model
scores
│   └─ demo_cases.csv                  # Genuine recorded examples, including
misses
│   └─ training_background.csv          # Actual training references for SHAP
│   └─ global_explanation_sample.csv
│   └─ global_shap.npz                 # Real, safely serialized SHAP values
│   └─ global_shap_metadata.json        # Model/reference hashes and additivity
checks
├─ docs/PAYSIM_PRESENTATION.md
├─ tests/test_presentation*.py
├─ card_dashboard.py / card_core.py    # Preserved separate card-history
experiment
├─ card_artifacts/                     # Its fitted model and demonstration data
├─ fraud_core.py / artifacts/          # Preserved original PCA notebook
experiment
├─ paysim_core.py / transfer_artifacts/ # Preserved earlier six-input transfer
study
└─ legacy_dashboard.py                  # Previous five-page research dashboard

```

The original notebooks, export code, and previous experiments remain available. `streamlit run legacy_dashboard.py` opens the earlier five-page interface. Its models and results remain separate from the new PaySim comparison.

Reproduce training and SHAP

Obtain the exact full CSV described in [the model bundle documentation](#), verify its SHA-256, and keep it outside the repository. The exported source-verification metadata supplies the audit file:

```
python prepare_presentation_model.py --data /path/outside/repository/paysim.csv
--source-manifest /path/source-audit.json --output /path/paysim-rebuild
--sample-size 500000 --seed 42
python presentation_explain.py /path/paysim-rebuild
```

The trainer checks the source fingerprint, derives activity before sampling, fits the three real models, exports the validation-selected pipeline, and checks reload prediction parity. The SHAP export checks additivity against the estimator and ties the stored explanation to model/background fingerprints. Changing the recipe creates a new experiment and requires a complete matching export.

Streamlit Community Cloud

The existing public app uses GitHub `main` and `app.py`. Pushing the complete committed bundle updates that deployment.

For a new deployment, sign in at [Streamlit Community Cloud](#), connect GitHub, choose this repository, branch `main`, and entry point `app.py`; select Python 3.12 in advanced settings and deploy. No API keys are required. Keep the pinned requirements and every presentation asset together. Open the resulting public URL on a phone or college computer.

The full raw 471-MB source is not deployed. The compact model/data/evaluation bundle is about 41 MB, with each individual file below GitHub's regular file-size limit. Only the selected model is loaded for runtime scoring; the comparison uses recorded outputs from all fitted candidates.